

ERROR HANDLING

I. Lesson

Why Errors Matter

- Errors are inevitable in programming.
- Rust emphasizes safety and reliability.
- Rust has **no exceptions**, unlike some languages.

Two Main Types of Errors

Error Type	How to Handle	Example
Recoverable Errors	<code>Result<T, E></code>	File not found, invalid input
Unrecoverable Errors	<code>panic!</code>	Array index out of bounds

Recoverable Errors



```
use std::fs::File;

fn main() {
    let f = File::open("hello.txt");
    let f = match f {
        Ok(file) => file,
        Err(error) => panic!("Problem opening the file: {:?}", error),
    };
}
```

Unrecoverable Errors



```
fn main() {  
    let v = vec![1, 2, 3];  
    v[99]; // This will panic!  
}
```


Error Propagation (?)



```
use std::fs::File;
use std::io::Read;

fn read_file() -> Result<String, std::io::Error> {
    let mut file = File::open("hello.txt");
    let mut contents = String::new();
    file.read_to_string(&mut contents);
    Ok(contents)
}
```

Best Practices

- Prefer Result for recoverable errors.
- Use panic! only for unrecoverable situations
- Add context to errors using crates like thiserror

The crate thiserror

- A Rust crate that makes defining custom error types easier.
- Automatically implements `std::error::Error` and `Display`.
- Works seamlessly with `Result<T, E>`.

```
use thiserror::Error;

#[derive(Error, Debug)]
pub enum MyError {
    #[error("File not found: {0}")]
    FileNotFound(String),

    #[error("Invalid input: {0}")]
    InvalidInput(String),
}
```



```
fn read_file(name: &str) -> Result<String, MyError> {
    if name == "missing.txt" {
        return Err(MyError::FileNotFound(name.to_string()));
    }
    Ok("file content".to_string())
}
```


II. Exercices

Pattern matching



```
match self {  
    NetworkError::Timeout => write!(f, "Request timed out"),  
    NetworkError::ConnectionRefused => write!(f, "Connection refused"),  
    NetworkError::InvalidUrl => write!(f, "Invalid URL"),  
}
```

The anyhow crate

```
● ● ●  
  
// WITHOUT anyhow - using String for errors  
fn parse_age_without(input: &str) -> Result<i32, String> {  
    let age = input  
        .parse::<i32>()  
        .map_err(|err| format!("Failed to parse: {}", err))?;  
  
    if age < 0 || age > 150 {  
        return Err("Age must be between 0 and 150".to_string());  
    }  
  
    Ok(age)  
}
```

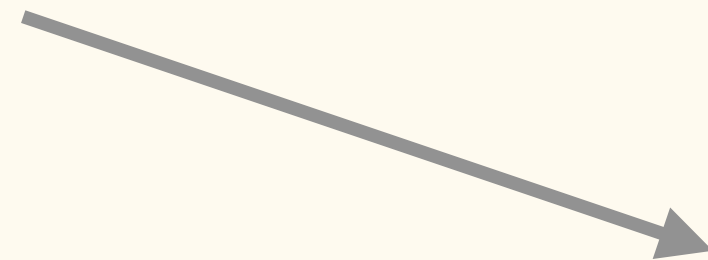
Without

With

```
● ● ●  
  
// WITH anyhow - simpler and better error context  
fn parse_age_with(input: &str) -> Result<i32> {  
    let age = input.parse::<i32>().context("Failed to parse"?);  
  
    if age < 0 || age > 150 {  
        anyhow::bail!("Age must be between 0 and 150");  
    }  
  
    Ok(age)  
}
```

The thiserror crate

```
#[derive(Error, Debug)]
enum FileError {
    #[error("File '{0}' not found")]
    NotFound(String),
    #[error("File '{0}' is too large: {1} bytes")]
    TooLarge(String, u64),
}
```



```
fn check_file(filename: &str, size: u64) -> Result<String, FileError> {
    if filename.is_empty() {
        return Err(FileError::NotFound(filename.to_string()));
    }

    if size > 1000 {
        return Err(FileError::TooLarge(filename.to_string(), size));
    }

    Ok(format!("File {} is valid", filename))
}
```


Don't forget to return early!

```
fn validate_user(username: &str, email: &str, age: i32) -> Result<String, String> {  
    // Check username - return immediately if invalid  
    if username.len() < 3 {  
        return Err("Username too short".to_string());  
    }  
  
    // Check email - return immediately if invalid  
    if !email.contains('@') {  
        return Err("Email invalid".to_string());  
    }  
  
    // Check age - return immediately if invalid  
    if age < 18 {  
        return Err("Age too low".to_string());  
    }  
  
    // All checks passed  
    Ok("User is valid".to_string())  
}
```

Chaining

```
fn main() {  
    // and_then: chain if success  
    let result1 = parse("5").and_then(double);  
    println!("{:?}", result1); // Ok(10)  
  
    let result2 = parse("abc").and_then(double);  
    println!("{:?}", result2); // Err("Invalid")  
  
    // or_else: fallback if error  
    let result3 = parse("xyz").or_else(|_| Ok::<i32, String>(0));  
    println!("{:?}", result3); // Ok(0)  
}
```


Let's code!

Your turn!